Introduction to Loops

Repeat work without repeating code: for, while and range

Why loops?

A loop runs a block of code many times. Less typing, fewer bugs, and the same code handles 3 items or 3 million.

```
for i in range(3):
    print("Hello", i)
```

OUTPUT

```
Hello 0
Hello 1
Hello 2
```

range(): numbers on demand

`range(stop)`, `range(start, stop)` or `range(start, stop, step)`. The stop value is **never** included.

```
list(range(5))              # → [0, 1, 2, 3, 4]
list(range(2, 6))           # → [2, 3, 4, 5]
list(range(0, 10, 3))       # → [0, 3, 6, 9]
list(range(5, 0, -1))       # → [5, 4, 3, 2, 1]
```

Accumulator pattern

Start with a neutral value, update it inside the loop, use it after the loop.

```
total = 0
for n in range(1, 6):
    total += n          # 1+2+3+4+5
print(total)
```

OUTPUT

```
15
```

Nested loops

The inner loop runs completely for every pass of the outer loop.

```
for row in range(1, 4):
    line = ""
    for col in range(1, 4):
        line += f"{row * col:3}"
    print(line)
```

OUTPUT

```
  1  2  3
  2  4  6
  3  6  9
```

Common mistakes

Wrong	Right
`range(1, 5)` expecting 5	stop is excluded: `range(1, 6)`
`for i in range(len(x))`	`for item in x:`
changing the list inside its own loop	loop over a copy
missing `:` or indent	`for x in y:` then 4 spaces

Two kinds of loop

Loop	Use it when
`for`	you have a collection, or you know how many times to repeat
`while`	you repeat until a condition changes (input, retries, games)

for loop anatomy

`item` takes each value in turn; the indented body runs once per value. The loop ends when the values run out.

```
for item in iterable:     # list, str, dict, range...
    ...                   # body: once per item
```

What can a for loop walk through?

Object	Each pass gives
`list`, `tuple`, `set`	the next item
`str`	the next character
`dict`	the next key (`.values()`, `.items()` for more)
`range(n)`	the next integer
open file	the next line

Find the maximum

```
nums = [4, 9, 2]
biggest = nums[0]
for n in nums:
    if n > biggest:
        biggest = n
print(biggest)
```

OUTPUT

```
9
```

The loop variable and _

```
for i in range(3):
    pass                  # empty body placeholder
print(i)                  # → 2
for _ in range(2):        # _ = value not needed
    print("hi")
```

OUTPUT

```
2
hi
hi
```

REMEMBER

- `range(n)` yields `0 ... n-1`: exactly `n` values, starting at 0.
- Indentation defines the loop body: dedent to end the loop.
- Choose `for` when you know the items or the count; `while` when you wait for a condition.
- Name the loop variable well: `for student in students`, not `for x in y`.
- The loop variable still exists after the loop, holding the last value.
- Runaway loop? Press Ctrl+C to stop the program.